

LAPORAN PRAKTIKUM STRUKTUR DATA

Merge, Quick dan Shell Sorting



Oleh :

MUHAMMAD ARIF

NIM 2511532017

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 29 Mei 2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan praktikum **“Merge, Quick dan Shell Sorting”** ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai dokumentasi dan pemahaman hasil kegiatan praktikum yang bertujuan untuk memahami bagaimana konsep dari Merge, Quick dan Shell Sorting. Penulis juga terbuka untuk menerima saran dan kritik guna perbaikan di masa mendatang.

Penulis mengucapkan terima kasih sebesar-besarnya kepada:

1. DR. Wahyudi, S.T, M.T selaku pembimbing mata kuliah struktur data, yang telah memberikan bimbingan, ilmu, dan motivasi selama proses pembelajaran.

Padang, 29 Mei 2026

Muhammad Arif

DAFTAR PUSTAKA

LAPORAN PRAKTIKUM STRUKTUR DATA.....	i
KATA PENGANTAR.....	ii
DAFTAR PUSTAKA	iii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN	3
2.1 Landasan teori	3
2.2 Langkah pengerjaan.....	4
BAB III KESIMPULAN.....	24
DAFTAR PUSTAKA	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital sekarang, pengelolaan dan pengolahan data menjadi fondasi utama dalam pengembangan sistem komputasi modern. Salah satu operasi yang penting dalam pengolahan data adalah pengurutan (sorting), yang berperan penting dalam mengorganisir kumpulan data agar lebih mudah dicari, dianalisis dan diproses oleh algoritma lain.

Sorting memiliki beragam pengurutan dengan kompleksitas waktu dan ruang yang beragam seperti BubbleSort, InsertionSort, MergeSort, QuickSort, dan SelectionSort. Setiap algoritma memiliki profil performa yang berbeda tergantung pada ukuran data, tingkat keterurutan awal, kebutuhan akan stabilitas pengurutan, serta batasan memori. Pemilihan algoritma yang tepat tidak hanya bergantung pada teori, tetapi juga pada konteks penggunaan nyata dan karakteristik data yang diolah.

Oleh karena itu, laporan praktikum ini dibuat sebagai dokumentasi sistematis dari proses implementasi, pengujian dan analisis algoritma sorting. Laporan ini bertujuan untuk memperkuat kompetensi praktikan dalam merancang solusi komputasi yang efisien.

1.2 Tujuan

Tujuan dari dilakukannya praktikum ini adalah

1. Memahami struktur Algoritma Merge, Quick dan Shell Sorting
2. Memahami langkah-langkah menulis Merge, Quick dan Shell Sorting

1.3 Manfaat

Manfaat yang diperoleh dari praktikum ini adalah

1. Meningkatkan pemahaman tentang Merge, Quick dan Shell Sorting
2. Meningkatkan pemahaman bagaimana cara menulis Merge, Quick dan Shell Sorting

BAB II

PEMBAHASAN

2.1 Landasan teori

Sorting merupakan ilmu komputer yang bertujuan untuk menyusun menyusun ulang elemen-elemen dalam suatu koleksi data berdasarkan nilai kunci tertentu, umumnya dalam urutan menaik (ascending) atau menurun (descending). Proses ini tidak hanya mempermudah pencarian data, tetapi juga menjadi prasyarat bagi banyak algoritma lain seperti pencarian biner, penggabungan data (merge), dan kompresi.

MergeSort merupakan algoritma pengurutan yang bekerja dengan membagi array secara rekursi menjadi dua bagian hingga setiap subarray hanya berisi satu elemen, kemudian menggabungkan subarray tersebut kembali secara berurutan hingga terbentuk array yang terurut. Algoritma ini memiliki kompleksitas waktu yang konsisten sebesar $O(n \log n)$ untuk kasus terbaik, rata-rata, maupun terburuk, di mana n adalah jumlah elemen. Dari segi penggunaan memori.

QuickSort mengadopsi divide and conquer seperti merge tetapi dengan metode yang berbeda. Algoritma ini memilih satu elemen sebagai pivot, lalu mempartisi array sehingga seluruh elemen lebih kecil dari pivot ditempatkan di sebelah kiri dan yang besar dari pivot di sebelah kanan. Quick Sort memiliki kompleksitas waktu terbaik dan rata-rata sebesar $O(n \log n)$, namun dapat menurun menjadi $O(n^2)$ pada kasus terburuk jika pemilihan pivot tidak optimal, misalnya pada data yang sudah terurut dengan strategi pivot yang naif. Kompleksitas ruangnya rata-rata $O(\log n)$ untuk tumpukan rekursi, dengan kasus terburuk $O(n)$.

ShellSort merupakan pengembangan dari InsertionSort yang bekerja dengan membandingkan dan menukar elemen-elemen yang berjarak tertentu (gap), bukan hanya elemen yang berdekatan. Algoritma ini dimulai dengan *gap* yang besar untuk melakukan penyortiran parsial, lalu secara bertahap mengurangi nilai *gap* hingga mencapai 1, di mana tahap terakhir secara efektif menjalankan Insertion Sort biasa. Ditemukan oleh Donald Shell, algoritma ini sering disebut juga sebagai diminishing increment sort. Kompleksitas waktu Shell Sort sangat bergantung pada urutan *gap* yang digunakan dengan urutan asli, kompleksitas terburuknya adalah $O(n^2)$.

2.2 Langkah pengerjaan

- MergeSort_2511532017
 1. Buka eclipse dan buat package baru dengan nama pekan8_2511532017 dan class MergeSort_2511532017.
 2. Setelah dibuat, kita akan membuat algoritma MergeSort

```
package Pekan8_2511532017;  
  
public class MergeSort_2511532017 {  
    void merge_2017 (int arr_2017[], int l_2017, int m_2017, int r_2017) {
```

3. Deklarasi metode merge_2017 untuk menggabungkan dua subarray yang sudah terurut menjadi satu array yang lebih besar dan tetap terurut, arr_2017[] itu array utama, l_2017 indeks kiri, m_2017 indeks tengah, r_2017 untuk indeks kanan.

```
//find size of two subarray to be merged  
int n1_2017 = m_2017 - l_2017 + 1;  
int n2_2017 = r_2017 - m_2017;  
//create temp array  
int L[] = new int[n1_2017];  
int R[] = new int[n2_2017];  
//copy data to temp array
```

4. Inisialisasi untuk menghitung ukuran panjang subarray kiri (n1) dan kanan(n2), lalu membuat array sementara L dan R untuk menampung data agar proses penggabungan tidak menimpa data asli yang belum diproses

```
//copy data to temp array  
for (int i_2017 = 0; i_2017 < n1_2017; ++i_2017)  
    L[i_2017] = arr_2017[l_2017 + i_2017];  
for (int j_2017 = 0; j_2017 < n2_2017; ++j_2017)  
    R[j_2017] = arr_2017[m_2017 + 1 + j_2017];  
int i_2017 = 0, j_2017 = 0;  
//initial index of merged subarray array
```

5. Loop pertama untuk menyalin elemen bagian kiri ke array L, loop kedua untuk menyalin bagian kanan dimulai dari m_2017 + 1 ke array R

```

int i_2017 = 0, j_2017 = 0;
//initial index pg merged subarray array
int k_2017 = 1_2017;
while ( i_2017 < n1_2017 && j_2017 < n2_2017) {
    if (L [i_2017] <= R[j_2017]) {
        arr_2017[k_2017] = L[i_2017];
        i_2017++;
    }else {
        arr_2017[k_2017] = R[j_2017];
        j_2017++;
    }
    k_2017++;
}

```

6. Proses penggabungan, dengan membandingkan elemen terkecil yang tersisa di L dan R, elemen yang lebih kecil disalin kembali ke arr_2017 pada posisi k_2017, indeks pengakses (i,j,k) dinaikkan untuk memproses elemen berikutnya

```

    k_2017++;
}
while (i_2017 < n1_2017) {
    arr_2017[k_2017] = L[i_2017];
    i_2017++;
    k_2017++;
}
while (j_2017 < n2_2017) {
    arr_2017[k_2017] = R[j_2017];
    j_2017++;
    k_2017++;
}
}

```

7. Menyalin sisa elemen, jika salah satu array (L atau R) masih memiliki sisa elemen setelah loop utama selesai, maka blok ini menyalin semua sisa ke array utama

```

void sort_2017 (int arr_2017[], int l_2017, int r_2017) {
    if(l_2017 < r_2017) {
        int m_2017 = (l_2017 + r_2017) / 2;
        sort_2017 (arr_2017, l_2017, m_2017);
        sort_2017 (arr_2017, m_2017 + 1, r_2017);
        merge_2017 (arr_2017, l_2017, m_2017, r_2017);
    }
}

```

8. Method divide and conquer, if memastikan pembagian hanya terjadi jika masih ada lebih dari satu elemen, divide $\text{int } m_2017 = (l_2017 + r_2017) / 2$; mencari titik tengah array, conquer untuk bagian kiri dan untuk bagian kanan, lalu menggabungkan kedua bagian yang sudah terurut tersebut.

```

static void printArray_2017 (int arr_2017[]) {
    int n_2017 =arr_2017.length;
    for (int i_2017 = 0; i_2017 < n_2017; ++i_2017)
        System.out.print(arr_2017[i_2017] + " ");
    System.out.println();
}

```

9. printArray_2017 untuk mencetak isi array

```
public static void main (String [] args) {  
    int arr_2017[] = {12,11,13,5,6,7};  
    System.out.print("sebelum di urutkan: ");  
    printArray_2017(arr_2017);  
    MergeSort_2511532017 ob_2017 = new MergeSort_2511532017();  
    ob_2017.sort_2017(arr_2017, 0, arr_2017.length-1);  
    System.out.print("\nSesudah terurut menggunakan mergeSort: ");  
    printArray_2017(arr_2017);  
}
```

10. method main, inialisasi data array {12,11,13,5,6,7}, cetak awal sebelum array terurut, ob_2017 digunakan untuk mengakses method non-static, memanggil sort_2017 dengan parameter awal 0 dan indeks akhir arr_2017.length-1, cetak hasil akhir yang terurut.

11. Output

```
sebelum di urutkan: 12 11 13 5 6 7  
  
Sesudah terurut menggunakan mergeSort: 5 6 7 11 12 13
```

- QuickSort_2511532017

1. Kita akan membuat class baru untuk QuickSort_2511532017

```
1 package Pekan8_2511532017;  
2  
3 public class QuickSort_2511532017 {  
4     static void swap_2017 (int [] arr_2017, int i_2017, int j_2017) {  
5         int temp_2017 = arr_2017[i_2017];  
6         arr_2017[i_2017] = arr_2017[j_2017];  
7         arr_2017[j_2017] = temp_2017;  
8     }  
9 }
```

2. Swap_2017 metode helper untuk menukar posisi dua elemen dalam array, i_2017 dan j_2017 merupakan dua elemen yang akan dipertukarkan, menggunakan temp_2017 untuk menyimpan nilai salah satu elemen agar tidak hilang saat proses penyimpanan

```
9 //metode tambahan untuk mengatur pivot menggunakan Median-of-Three  
10 static void medianOfThree_2017 (int[] arr_2017, int low_2017, int high_2017) {  
11     int mid_2017 = low_2017 + (high_2017 - low_2017)/2;  
12  
13     //urutkan elemen low, mid, dan high  
14     if (arr_2017[low_2017] > arr_2017[mid_2017]) {  
15         swap_2017 (arr_2017, low_2017, mid_2017);  
16     }  
17     if (arr_2017[low_2017] > arr_2017[high_2017]) {  
18         swap_2017 (arr_2017, low_2017, high_2017);  
19     }  
20     if (arr_2017[mid_2017] > arr_2017[high_2017]) {  
21         swap_2017 (arr_2017, mid_2017, high_2017);  
22     }  
23     swap_2017 (arr_2017, mid_2017, high_2017);  
24 }
```

3. Method medianOfThree_2017 untuk memilih pivot dari tiga kandidat elemen pertama (low), tengah (mid), terakhir (high), pertama hitung indeks tengah, if pertama mengurutkan nilai pada

indeks low, mid dan high sehingga $\text{arr}[\text{low}] \leq \text{arr}[\text{mid}] \leq \text{arr}[\text{high}]$, posisi pivot `swap_2017(arr_2017, mid_2017, high_2017)` agar pindah ke posisi high.

```
static int partition_2017 (int[] arr_2017, int low_2017, int high_2017) {
    //panggil fungsi medianOfThree sebelum mengurutkan pivot
    medianOfThree_2017(arr_2017, low_2017, high_2017);

    int pivot_2017 = arr_2017[high_2017];
    int i_2017 = (low_2017 - 1);

    for (int j_2017 = low_2017; j_2017 <= high_2017-1; j_2017++) {
        if( arr_2017[j_2017] < pivot_2017) {
            i_2017++;
            swap_2017 (arr_2017, i_2017, j_2017);
        }
    }
    swap_2017 (arr_2017, i_2017+1, high_2017 );
    return (i_2017 + 1);
}
```

4. Method `partition_2017` memanggil `medianOfThree_2017` dan menetapkan `pivot_2017` sebagai elemen indeks high, `i_2017` penanda batas akhir dari elemen-elemen yang lebih kecil dari pivot diinisialisasi di `low-1`, `j_2017` iterasi melalui array, loop jika `arr[j] < pivot` elemen harus pindah ke kiri, `i_2017` dinaikkan, lalu swap antara `arr[i]` dan `arr[j]`, `swap_2017(arr_2017, i_2017+1, high_2017)` memindahkan pivot dari posisi high ke posisi sebenarnya (`i+1`)

```
static void QuickSort_2511532017 (int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition_2017 (arr, low, high);
        QuickSort_2511532017 (arr, low, pi - 1);
        QuickSort_2511532017(arr, pi+1, high);
    }
}
```

5. Method `QuickSort_2511532017`, if (`low < high`) memastikan rekursi berhenti jika subarray memiliki 0 atau 1 elemen, memanggil `partition_2017` untuk menempatkan pivot di posisi yang benar (`pi`), rekursi kiri mengurutkan elemen dari `low` sampai `pi-1`, bagian kanan mengurutkan elemen dari `pi+1` sampai `high`

```
public static void printArr_2017 (int [] arr_2017) {
    for (int i_2017 = 0; i_2017< arr_2017.length; i_2017++) {
        System.out.print(arr_2017[i_2017]+" ");
    }
    System.out.println();
}
```

6. Method `print` untuk mencetak array

```

public static void main (String []args) {
    int [] arr_2017 = {10,7,8,9,1,5};
    int N = arr_2017.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2017(arr_2017);

    QuickSort_2511532017 (arr_2017, 0, N-1);

    System.out.print("data setelah diurutkan dengan qucksrot: ");
    printArr_2017(arr_2017);
}
}

```

7. Method main, inialisasi array {10,7,8,9,1,5}, cetak awal sebelum diurutkan , pemanggilan sorting QuickSort_2511532017(arr_2017, 0, N-1) memulai proses pengurutan lalu cetak hasil
8. Output

```

<terminated> QuickSort_2511532017 [Java Application] C:\Users\User\p2\p
Data sebelum diurutkan: 10 7 8 9 1 5
data setelah diurutkan dengan qucksrot: 1 5 7 8 9 10

```

- ShellSort_2511532017

1. kita akan membuat class baru untuk ShellSort_2511532017

```

package Pekan8_2511532017;

public class ShellSort_2511532017 {
    public static void ShellSort_2511532017 (int [] A_2017){
        int n_2017 = A_2017.length;
        int gap_2017 = n_2017/2;
        while (gap_2017 > 0) {

```

2. method ShellSort_2511532017 merupakan method untuk algoritma ShellSort, n_2017 menyimpan panjang array, gap_2017 = n_2017/2 menginisialisasi nilai gap (jarak) awal, loop akan berjalan sampai gap masih besar dari 0, setiap kali loop ini selesai, nilai gap_2017 akan diperkecil

```

        for (int i_2017 = gap_2017; i_2017 < n_2017; i_2017++) {
            int temp_2017 = A_2017[i_2017];
            int j_2017 = i_2017;
            while ( j_2017 >= gap_2017 && A_2017[j_2017 - gap_2017] > temp_2017) {
                A_2017[j_2017] = A_2017[j_2017-gap_2017];
                j_2017 = j_2017 - gap_2017;
            }
            A_2017[j_2017] = temp_2017;
        }
        gap_2017 = gap_2017/2;
    }
}

```

3. loop dimulai dari dari indeks gap_2017 hingga akhir array, temp_2017 menyimpan nilai elemen saat ini A[i] yang akan

disisipkan ke posisi yang tepat, j_2017 sebagai pointer untuk menelusuri mundur elemen dengan lompatan sejauh gap

```
while ( j_2017 >= gap_2017 && A_2017[j_2017 - gap_2017] > temp_2017) {  
    A_2017[j_2017] = A_2017[j_2017-gap_2017];  
    j_2017 = j_2017 - gap_2017;  
}  
A_2017[j_2017] = temp_2017;  
}
```

4. proses pergeseran, loop while j_2017 >= gap_2017 memastikan indeks tidak keluar dari batas array bagian kiri, A_2017[j_2017 - gap_2017] > temp_2017 membandingkan elemen di kiri (dengan jarak gap) dengan temp, jika elemen lebih besar maka digeser, A_2017[j_2017] = A_2017[j_2017-gap_2017] menggeser elemen yang lebih besar ke posisi kanan, A_2017[j_2017] = A_2017[j_2017-gap_2017] memindahkan pointer j mundur sejauh gap untuk membandingkan dengan elemen berikutnya. Setelah ditemukan posisi yang tepat, nilai temp_2017 ditempatkan pada indeks j_2017

```
}  
gap_2017 = gap_2017/2;  
}  
}
```

5. setelah seluruh array diproses dengan gap saat ini, maka gap_2017 dibagi 2 untuk iterasi berikutnya, proses berlanjut hingga gap_2017 menjadi 0

```
public static void main (String []args) {  
    int [] data_2017 = {3,10,4,6,8,9,7,2,1,5};  
  
    System.out.print("Sebelum: ");  
    printArray_2017(data_2017);  
  
    ShellSort_2511532017(data_2017);  
  
    System.out.print("Setelah (ShellSort): ");  
    printArray_2017(data_2017);  
}
```

6. method main inialisasi array dengan nilai {3,10,4,6,8,9,7,2,1,5}, cetak awal sebelum diurutkan, eksekusi memanggil method ShellSort_2511532017(data_2017) untuk mengurutkan array, cetak hasil setelah diurutkan

```

    public static void printArray_2017 (int [] arr_2017) {
        for (int i_2017 : arr_2017) System.out.print(i_2017 + " ");
        System.out.println();
    }
}

```

7. method print untuk mencetak hasil
8. output

```

Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (ShellSort): 1 2 3 4 5 6 7 8 9 10

```

- BubbleGUI_2511532017
 1. Buat class dengan nama BubbleGUI_2511532017

```

package Pekan8_2511532017;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class BubbleSortGUI_2511532017 extends JFrame {

    private JTextField inputField_2017;
    private JButton setButton_2017;
    private JButton stepButton_2017;
    private JButton resetButton_2017;
    private JTextArea stepArea_2017;
    private JPanel panelArray_2017;

    private int[] array_2017;
    private JLabel[] labelArray_2017;

    private int i_2017 = 0;
    private int j_2017 = 0;
    private int stepCount_2017 = 1;
    private boolean sorting_2017 = false;
}

```

2. Import java.awt.* untuk komponen dasar GUI (layout, warna, font), import javax.swing.* untuk komponen dasar seperti JFrame, JButton dan SwingUtilities untuk thread safety

```

public BubbleSortGUI_2511532017() {
    setTitle("Visualisasi Bubble Sort");
    setSize(800, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout(10, 10));
}

```

3. Konstruktor untuk jendela utama

```
// Panel input bagian atas
JPanel inputPanel_2017 = new JPanel(new FlowLayout());

JLabel inputLabel_2017 = new JLabel("Masukkan angka (pisahkan dengan koma):");
inputField_2017 = new JTextField(30);

setButton_2017 = new JButton("Set Array");
stepButton_2017 = new JButton("Step");
resetButton_2017 = new JButton("Reset");

stepButton_2017.setEnabled(false);

inputPanel_2017.add(inputLabel_2017);
inputPanel_2017.add(inputField_2017);
inputPanel_2017.add(setButton_2017);
inputPanel_2017.add(stepButton_2017);
inputPanel_2017.add(resetButton_2017);

add(inputPanel_2017, BorderLayout.NORTH);
```

4. Panel input, untuk membuat panel horizontal dibagian atas untuk input data dan kontrol tombol, `stepButton_2017.setEnabled(false)` tombol step dinonaktifkan secara default

```
// Panel untuk menampilkan array
panelArray_2017 = new JPanel(new FlowLayout());
panelArray_2017.setPreferredSize(new Dimension(750, 120));
add(panelArray_2017, BorderLayout.CENTER);
```

5. Area tengah untuk menampilkan elemen array sebagai JLabel yang diatur secara horizontal

```
// Text area untuk menampilkan langkah sorting
stepArea_2017 = new JTextArea();
stepArea_2017.setEditable(false);
stepArea_2017.setFont(new Font("Monospaced", Font.PLAIN, 14));

JScrollPane scrollPane = new JScrollPane(stepArea_2017);
scrollPane.setPreferredSize(new Dimension(750, 250));

add(scrollPane, BorderLayout.SOUTH);
```

6. Area log langkah untuk menampilkan riwayat langkah sorting

```
// Event tombol
setButton_2017.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        setArrayFromInput_2017();
    }
});

stepButton_2017.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        performStep_2017();
    }
});

resetButton_2017.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        reset_2017();
    }
});
}
```

7. Registrasi event listener menghubungkan setiap tombol dengan method yang akan dieksekusi saat tombol diklik

```
private void setArrayFromInput_2017() {
    String text_2017 = inputField_2017.getText().trim();

    if (text_2017.isEmpty()) {
        return;
    }

    String[] parts_2017 = text_2017.split(",");
    array_2017 = new int[parts_2017.length];

    try {
        for (int k = 0; k < parts_2017.length; k++) {
            array_2017[k] = Integer.parseInt(parts_2017[k].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(
            this,
            "Masukkan hanya angka yang dipisahkan koma!",
            "Error",
            JOptionPane.ERROR_MESSAGE
        );
        return;
    }
}
```

8. Method setArrayFromInput_2017, membaca teks dan memisahkan berdasarkan koma dan mengonversi menjadi array integer, menggunakan try catch untuk menangani jika pengguna memasukkan karakter non angka

```
i_2017 = 0;
j_2017 = 0;
stepCount_2017 = 1;
sorting_2017 = true;

stepButton_2017.setEnabled(true);
stepArea_2017.setText("");
panelArray_2017.removeAll();

labelArray_2017 = new JLabel[array_2017.length];

for (int k = 0; k < array_2017.length; k++) {
    labelArray_2017[k] = new JLabel(String.valueOf(array_2017[k]));
    labelArray_2017[k].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_2017[k].setOpaque(true);
    labelArray_2017[k].setBackground(Color.WHITE);
    labelArray_2017[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
    labelArray_2017[k].setPreferredSize(new Dimension(50, 50));
    labelArray_2017[k].setHorizontalAlignment(SwingConstants.CENTER);

    panelArray_2017.add(labelArray_2017[k]);
}

panelArray_2017.revalidate();
panelArray_2017.repaint();
}
```

9. Mereset variabel kontrol sorting, mengaktifkan tombol step, membuat JLabel untuk setiap angka, memberikan gaya visual revalidate() dan repaint() memaksa GUI memperbarui tampilan
10. Method performStep_2017 mengeksekusi satu iterasi dari algoritma bubble sort setiap kali tombol step diklik

```
private void performStep_2017() {
    if (!sorting_2017 || i_2017 >= array_2017.length - 1) {
        sorting_2017 = false;
        stepButton_2017.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }
}
```

11. Pengecekan jika sorting sudah selesai atau indeks i sudah mencapai batas, tampilkan pesan siap

```
resetHighlights_2017();

StringBuilder stepLog_2017 = new StringBuilder();

labelArray_2017[j_2017].setBackground(Color.CYAN);
labelArray_2017[j_2017 + 1].setBackground(Color.CYAN);
```

12. Mengubah warna latar kedua elemen yang sedang dibandingkan menjadi cyan agar pengguna tahu elemen mana yang sedang diproses

```
if (array_2017[j_2017] > array_2017[j_2017 + 1]) {
    // Swap
    int temp = array_2017[j_2017];
    array_2017[j_2017] = array_2017[j_2017 + 1];
    array_2017[j_2017 + 1] = temp;

    labelArray_2017[j_2017].setBackground(Color.RED);
    labelArray_2017[j_2017 + 1].setBackground(Color.RED);

    stepLog_2017.append("Langkah ").append(stepCount_2017).append(": ")
        .append("Menukar elemen ke-")
        .append(j_2017)
        .append(" dengan ke-")
        .append(j_2017 + 1)
        .append(" dengan ke-")
        .append(j_2017 + 1)
        .append(" dengan ke-")
        .append(j_2017);
} else {
    stepLog_2017.append("Langkah ").append(stepCount_2017).append(": ")
        .append("Tidak ada pertukaran antara ke-")
        .append(j_2017)
        .append(" dan ke-")
        .append(j_2017 + 1);
}

stepLog_2017.append("Hasil: ").append(arrayToString_2017(array_2017)).append("\n\n");
stepArea_2017.append(stepLog_2017.toString());

updateLabels_2017();

j_2017++;
```

13. Jika elemen kiri > elemen kanan, lakukan penukaran data dan ubah warna label menjadi merah untuk menandakan terjadi swap, jika tidak biarkan tetap, log aktivitas dicatat di stepArea_2017

```
stepLog_2017.append("Hasil: ").append(arrayToString_2017(array_2017)).append("\n\n");
stepArea_2017.append(stepLog_2017.toString());

updateLabels_2017();

j_2017++;

if (j_2017 >= array_2017.length - 1 - i_2017) {
    j_2017 = 0;
    i_2017++;
}

stepCount_2017++;
```

14. Update tampilan teks pada labek jika ada nilai yang berubah, menaikkan j_2017 untuk perbandingan berikutnya, jika satu pass siap (j mencapai batas) reset j=0 dan naikan i_2017

```
if (i_2017 >= array_2017.length - 1) {
    sorting_2017 = false;
    stepButton_2017.setEnabled(false);
    resetHighlights_2017();

    for (JLabel label : labelArray_2017) {
        label.setBackground(Color.GREEN);
    }

    JOptionPane.showMessageDialog(this, "Sorting selesai!");
}
```

15. Jika seluruh proses selesai, ubah warna semua label menjadi hijau

```
private void updateLabels_2017() {
    for (int k_2017 = 0; k_2017 < array_2017.length; k_2017++) {
        labelArray_2017[k_2017].setText(String.valueOf(array_2017[k_2017]));
    }
}
```

16. updateLabels_2017() memastikan teks yang ditampilkan di JLabel selau sinkron dengan nilai array_2017

```
private void resetHighlights_2017() {
    for (JLabel label : labelArray_2017) {
        label.setBackground(Color.WHITE);
    }
}
```

17. resetHighlights_2017() mengembalikan warna latar semua label ke warna putih

```
private void reset_2017() {
    inputField_2017.setText("");
    panelArray_2017.removeAll();
    panelArray_2017.revalidate();
    panelArray_2017.repaint();

    stepArea_2017.setText("");
    stepButton_2017.setEnabled(false);

    sorting_2017 = false;
    i_2017 = 0;
    j_2017 = 0;
    stepCount_2017 = 1;
}
```

18. reset_2017() mengosongkan input, membersihkan panel array dan log serta mereset semua variabel kontrol ke nilai awal

```

private String arrayToString_2017(int[] arr_2017) {
    StringBuilder sb_2017 = new StringBuilder();

    for (int k_2017 = 0; k_2017 < arr_2017.length; k_2017++) {
        sb_2017.append(arr_2017[k_2017]);

        if (k_2017 < arr_2017.length - 1) {
            sb_2017.append(", ");
        }
    }

    return sb_2017.toString();
}

```

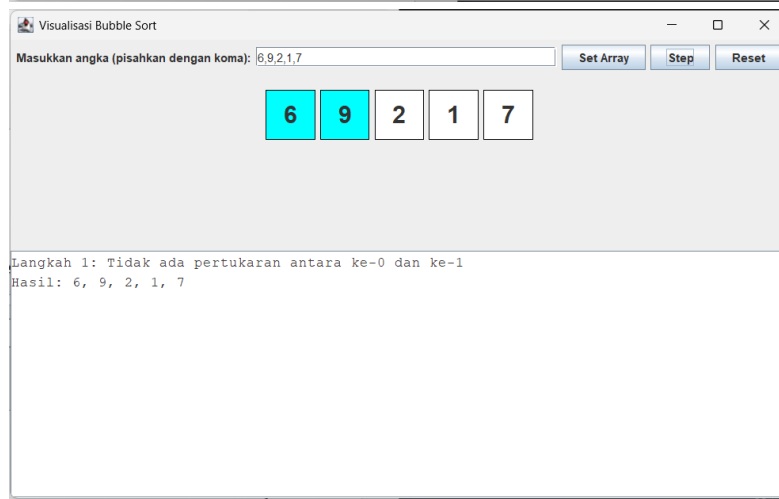
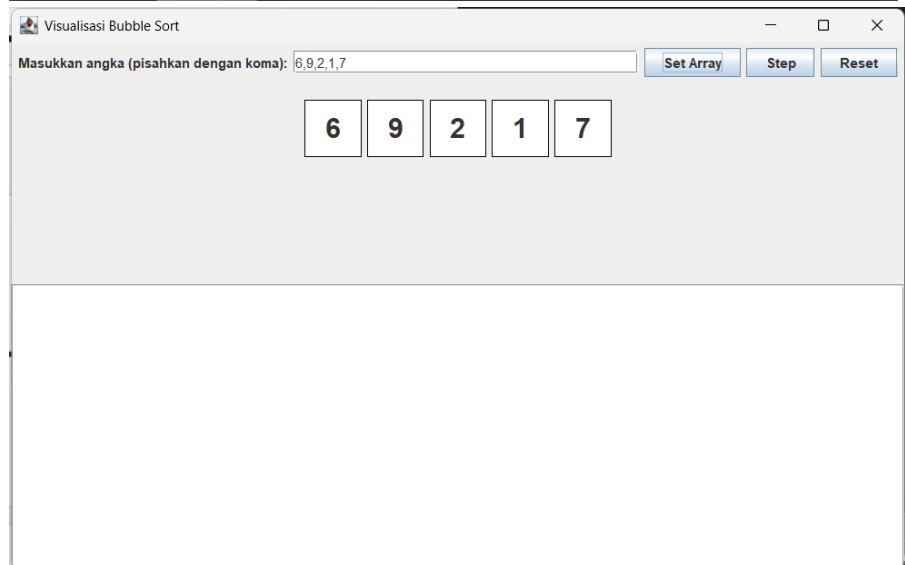
19. arrayToString_2017(int[] arr_2017) membantu mengubah array integer menjadi format string

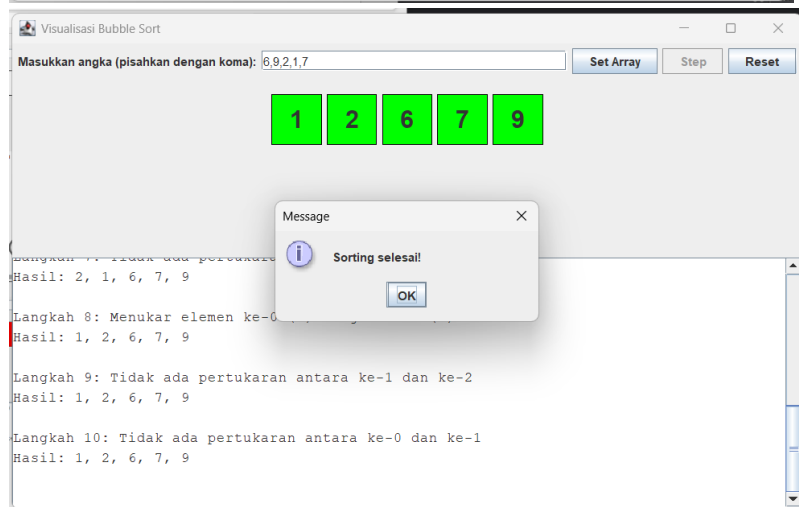
```

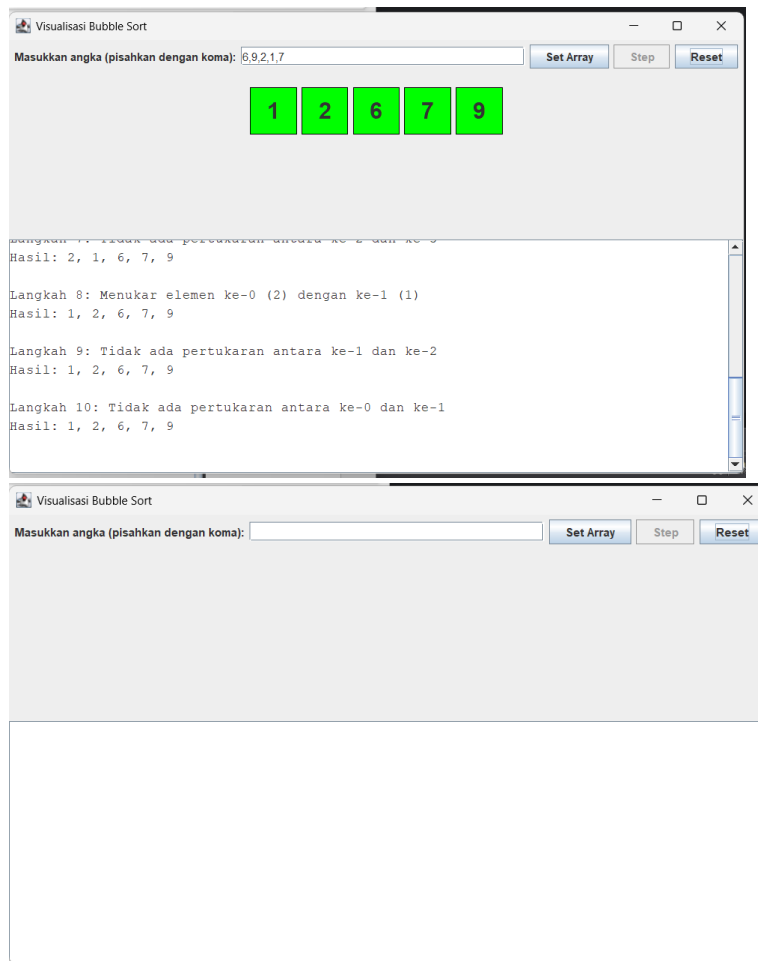
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new BubbleSortGUI_2511532017().setVisible(true);
        }
    });
}

```

20. method main, SwingUtilities.invokeLater memastikan bahwa pembuatan dan penampilan GUI dilakukan di Event Dispatch Thread (EDT)
21. output







Latihan

Pada latihan kali ini saya memilih ShellSort untuk algoritma sortingnya karena itu yang cukup simpel diimplementasikan. Lalu untuk kompleksitas terburuk untuk shellSort ini adalah $O(n^2)$ dan untuk best case $O(n \log n)$.

- Lagu_2511532017
 1. Kita akan membuat class baru untuk Lagu_2511532017

```
package Pekan8_2511532017;

public class Lagu_2511532017 {
    private String judul_2017;
    private String penyanyi_2017;
    private int durasi_2017;
    Lagu_2511532017(String judul_2017, String penyanyi_2017, int durasi_2017){
        this.judul_2017 = judul_2017;
        this.penyanyi_2017 = penyanyi_2017;
        this.durasi_2017 = durasi_2017;
    }
}
```

2. Deklarasi class dan atribut, private String judul_2017 untuk menyimpan judul, penyanyi_2017 untuk menyimpan nama penyanyi, durasi_2017 untuk menyimpan prodi dari durasi

```
package pekan7_2511532017;

public class MahasiswaSort_2511532017 {
    private String nim_2017;
    private String nama_2017;
    private String prodi_2017;
    MahasiswaSort_2511532017(String nim_2017, String nama_2017, String prodi_2017){
        this.nim_2017 = nim_2017;
        this.nama_2017 = nama_2017;
        this.prodi_2017 = prodi_2017;
    }
}
```

3. Constructor, method khusus yang dipanggil saat membuat objek Lagu_2511532017 baru, menginisialisasi judul dan penyanyi

```

    }
    public String Judul_2017() {return judul_2017;}
    public void Judul_2017(String judul_2017) {this.judul_2017 = judul_2017;}

    public String Penyanyi_2017() {return penyanyi_2017;}
    public void Penyanyi_2017(String penyanyi_2017) {this.penyanyi_2017 = penyanyi_2017;}

    public int Durasi_2017() {return durasi_2017;}
    public void Durasi_2017(int durasi_2017) {this.durasi_2017 = durasi_2017;}

    public String toString() {
        return "\nJudul : " + judul_2017 +
            "\nPenyanyi : " + penyanyi_2017 +
            "\nDurasi: " + durasi_2017;
    }
}
```

4. Getter untuk mengambil nilai atribut private diluar kelas dan setter untuk mengubah nilai atribut private di luar kelas

- Sorting_2511532017

1. Kita buat kelas untuk ShelSort_2511532017

```
1 package Pekan8_2511532017;
2
3 import java.util.Scanner;
4
5 public class ShelSort_2511532017 {
6     private static final int MAKS_LAGU = 20;
7 }
```

2. private static final int MAKS_LAGU = 20; konstanta untuk membatasi maksimal lagu

```

public static void shellSortJudul(Lagu_2511532017[] daftarLagu) {
    int n = daftarLagu.length;
    int gap = n / 2;

    while (gap > 0) {
        for (int i = gap; i < n; i++) {
            Lagu_2511532017 temp = daftarLagu[i];
            int j = i;

            // Bandingkan judul
            while (j >= gap && daftarLagu[j - gap].Judul_2017().compareToIgnoreCase(temp.Judul_2017()) > 0) {
                daftarLagu[j] = daftarLagu[j - gap];
                j -= gap;
            }
            daftarLagu[j] = temp;
        }
        gap /= 2;
    }
}

```

3. method ShellSort untuk mengurutkan judul lagu menggunakan compareToIgnoreCase

```

// Helper untuk mencetak playlist
public static void printPlaylist(Lagu_2511532017[] daftarLagu) {
    for (int i = 0; i < daftarLagu.length; i++) {
        System.out.println((i + 1) + ". " + daftarLagu[i].Judul_2017() + " - " + daftarLagu[i].Durasi_2017() + " detik");
    }
}

```

4. method printPlaylist untuk merapikan tampilan output

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("=== Sorting Playlist NIM: 2511532017 === \nmenggunakan ShellSort");
    System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): ");
    String inputPilihan = sc.nextLine().trim();

    int pilihan;
    try {
        pilihan = Integer.parseInt(inputPilihan);
    } catch (NumberFormatException e) {
        System.out.println("algoritma tidak tersedia");
        return;
    }

    if (pilihan != 1) {
        System.out.println("algoritma tidak tersedia");
        return;
    }
}

```

5. method main sc.nextLine().trim() membaca input user, try-catch menangani error, if (pilihan != 1) hanya mengizinkan pilihan 1.

```

// Input data lagu dari user
System.out.print("Masukkan jumlah lagu: ");
int jumlah = Integer.parseInt(sc.nextLine().trim());
if (jumlah < 1 || jumlah > MAKS_LAGU) {
    System.out.print(" jumlah lagu terlalu banyak");
    return;
} else {
    Lagu_2511532017[] daftarLagu = new Lagu_2511532017[jumlah];
}

Lagu_2511532017[] daftarLagu = new Lagu_2511532017[jumlah];

```

6. Input jumlah lagu jumlah < 1 || jumlah > MAKS_LAGU, tolak dan hentikan program

```

for (int i = 0; i < jumlah; i++) {
    System.out.println("");
    System.out.print("Judul: ");
    String judul = sc.nextLine();
    System.out.print("Penyanyi: ");
    String penyanyi = sc.nextLine();
    System.out.print("Durasi (detik): ");
    int durasi = Integer.parseInt(sc.nextLine().trim());
    System.out.println("");
    daftarLagu[i] = new Lagu_2511532017(judul, penyanyi, durasi);
}

```

7. Input detail setiap lagu

```

// Tampilkan data sebelum sorting
System.out.println("\nData Sebelum Sorting: ");
printPlaylist(daftarLagu);

shellSortJudul(daftarLagu);

// Tampilkan data setelah sorting
System.out.println("\nData Setelah Shell Sort (Judul Asc): ");
printPlaylist(daftarLagu);

sc.close();
}
}

```

8. Tampilan data sebelum sorting dan sesudah sorting

9. Output

```

=== Sorting Playlist NIM: 2511532017 ===
menggunakan ShellSort
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 1
Masukkan jumlah lagu: 7

Judul: feel it
Penyanyi: D4vid
Durasi (detik): 214

Judul: die very rough
Penyanyi: mario judah
Durasi (detik): 123

Judul: here with me
Penyanyi: D4vid
Durasi (detik): 341

Judul: Whistle
Penyanyi: Florida
Durasi (detik): 109

Judul: Best day of my life
Penyanyi: American author
Durasi (detik): 241

Judul: The Night
Penyanyi: Avicii
Durasi (detik): 211

```

```

Judul: Pompeii
Penyanyi: Bastille
Durasi (detik): 211

Data Sebelum Sorting:
1. feel it - 214 detik
2. die very rough - 123 detik
3. here with me - 341 detik
4. Whistle - 109 detik
5. Best day of my life - 241 detik
6. The Night - 211 detik
7. Pompeii - 211 detik

Data Setelah Shell Sort (Judul Asc):
1. Best day of my life - 241 detik
2. die very rough - 123 detik
3. feel it - 214 detik
4. here with me - 341 detik
5. Pompeii - 211 detik
6. The Night - 211 detik
7. Whistle - 109 detik

```

```

=== Sorting Playlist NIM: 2511532017 ===
menggunakan ShellSort
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 2
algoritma tidak tersedia

```

```

=== Sorting Playlist NIM: 2511532017 ===
menggunakan ShellSort
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge): 1
Masukkan jumlah lagu: 21
jumlah lagu terlalu banyak

```


BAB III

KESIMPULAN

Berdasarkan praktikum yang dilakukan, dapat disimpulkan bahwa implementasi algoritma dilakukan dengan baik. Pada latihan shellsort berhasil diterapkan dengan mengurutkan judul lagu menggunakan struktur data array. Shellsort menunjukkan keunggulan dalam menangani data berukuran kecil hingga menengah dengan efisiensi lebih baik dari InsertionSort berkat mekanisme gap yang memungkinkan perpindahan elemen jarak jauh untuk mempercepat pengurutan.

DAFTAR PUSTAKA

- [1] D. L. Shell, "A high-speed sorting procedure," *Communications of the ACM*, vol. 2, no. 7, pp. 30–32, Jul. 1959, doi: 10.1145/368370.368387.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 2, pp. 29–37.